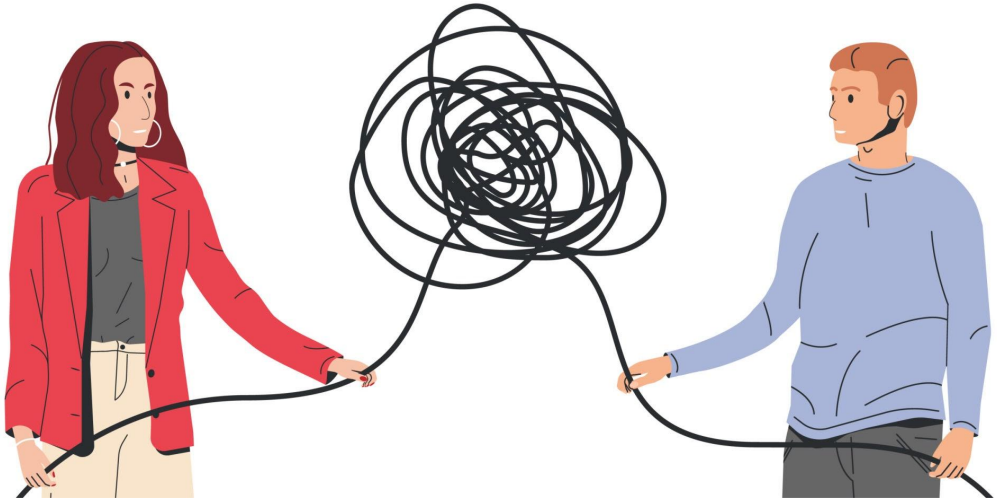
SOLID PRINCIPLES

Building Software that Lasts

The Problem: Bad Software Design

The code we fear to change.



- Rigid Code: A small change breaks everything.
- Fragile Code: Fixing one bug creates two more.
- Immobile Code: Can't reuse code because it's too entangled.
- Viscous Code: It's easier to write a hack than to do it right.

What is SOLID?

- A set of 5 design principles for Object-Oriented Programming.
- Coined by Robert C. Martin ("Uncle Bob").
- Goal: Create software that is:
  - Understandable
  - Flexible
  - Maintainable
  - Testable

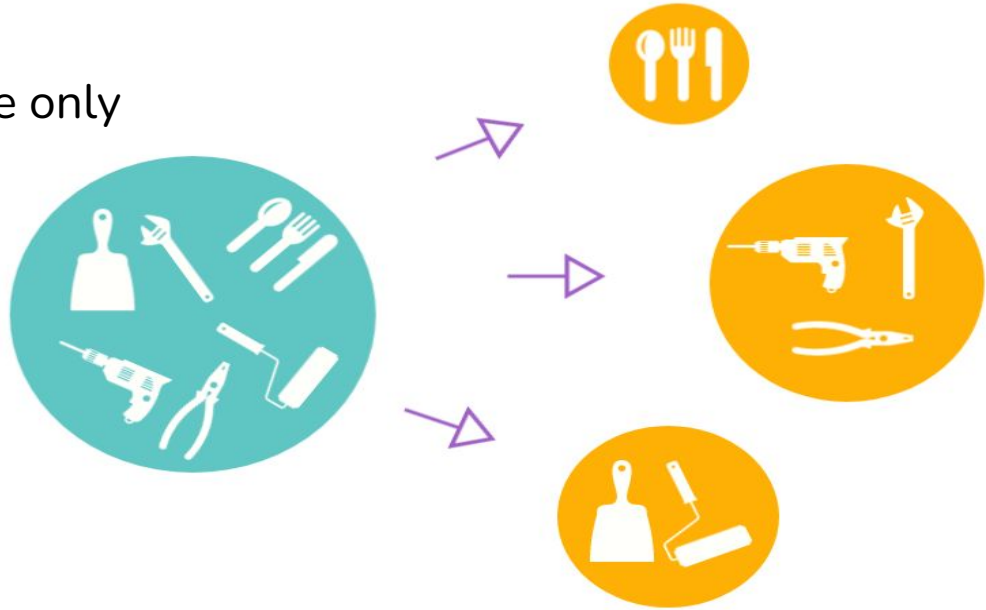

Single responsibility
S
O
Open-Closed Principle
Liskov substitution
L
I
Interface segregation
Dependency inversion
D

S: Single Responsibility Principle

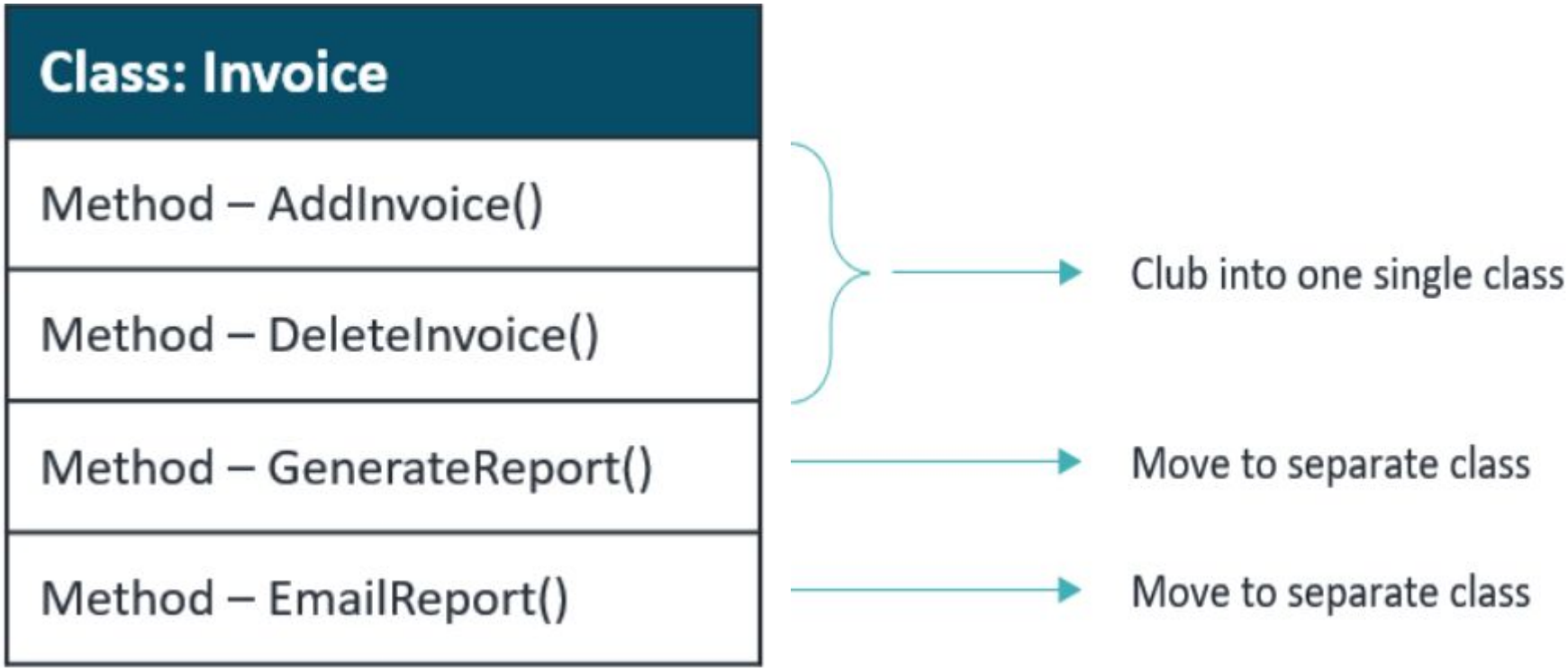
“A class should have only one job or responsibility.”

Single Responsibility Principle states:

1. Your class or method should have only one reason to change.
2. Your class or method should have only one responsibility to take care of.



Example


Class: Invoice
Method – AddInvoice()
Method – DeleteInvoice()
Method – GenerateReport()
Method – EmailReport()
Club into one single class
Move to separate class
Move to separate class

Identifying Violations of SRP

1. The class has too many methods (usually 10+)
2. Methods can be grouped by different themes (user management, email, reporting)
3. The class imports many unrelated dependencies
4. The class name is vague ("Manager", "Processor", "Handler")
5. Test files are huge and test many different behaviors

```java
public class UserManager {
    // Responsibility 1: User Data Management
    public void createUser(User user) { ... }
    public void updateUser(User user) { ... }
    public void deleteUser(int userId) { ... }

    // Responsibility 2: Authentication
    public boolean login(String username, String password) { ... }
    public void logout(User user) { ... }
    public void resetPassword(String email) { ... }

    // Responsibility 3: Email Handling
    public void sendWelcomeEmail(User user) { ... }
    public void sendPasswordResetEmail(User user) { ... }

    // Responsibility 4: Data Persistence
    public void saveToDatabase(User user) { ... }
    public User loadFromDatabase(int userId) { ... }
```

solution?

Break it down!

```java
// Responsibility 1: User Entity (just data)
public class User {
    private int id;
    private String username;
    private String email;
    // constructor, getters, setters
}
```

```java
// Responsibility 3: Authentication
public class AuthenticationService {
    private UserRepository userRepository;

    public boolean login(String username, String password) { ... }
    public void logout(User user) { ... }
    public void resetPassword(String email) { ... }
}
```

```java
// Responsibility 2: User Data Management
public class UserRepository {
    public void saveUser(User user) { ... }
    public User findUserById(int userId) { ... }
    public void updateUser(User user) { ... }
    public void deleteUser(int userId) { ... }
}
```

Common Misconceptions

Misconception 1: "One Method Per Class"

Misconception 2: "Only Applies to Classes”

SRP can (and should) be applied at multiple levels:

- Methods: Should do one thing
- Classes: Should have one responsibility
- Modules: Should handle one domain concern
- Services: Should provide one type of service

O: Open / Closed principle

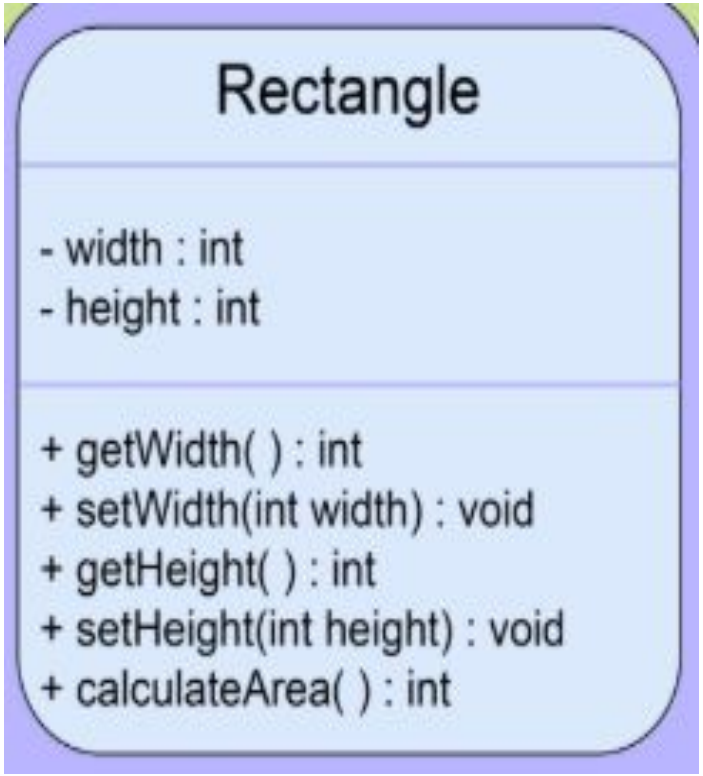
“software entities should be open for extension, but closed for modification.”

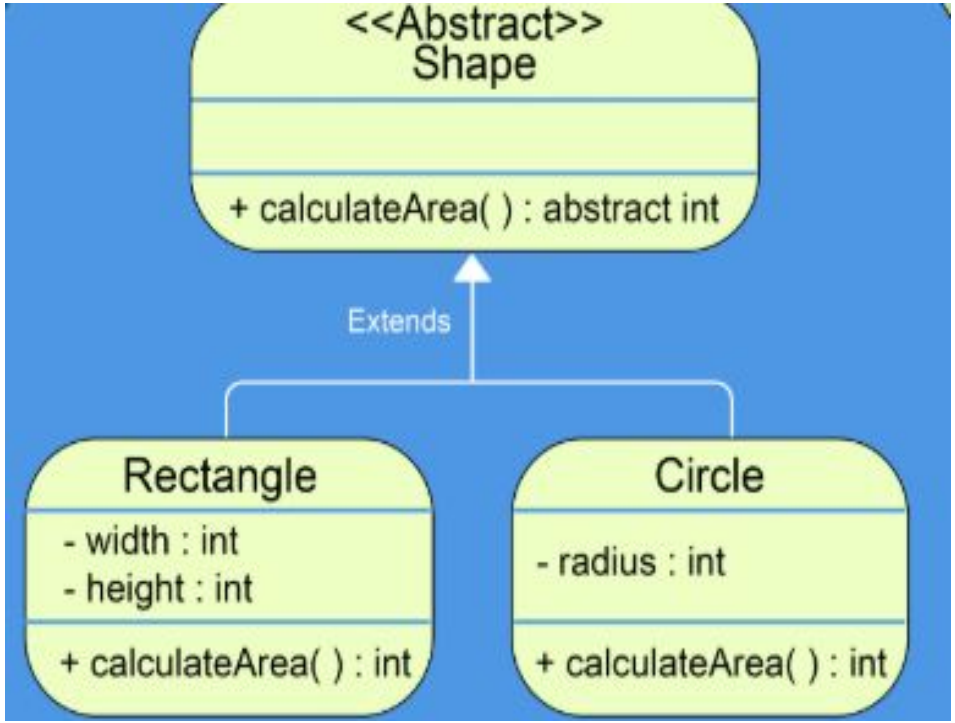
Open / Closed principle

It means you or your team members should be able to add new functionalities to an existing software system without changing the existing code.

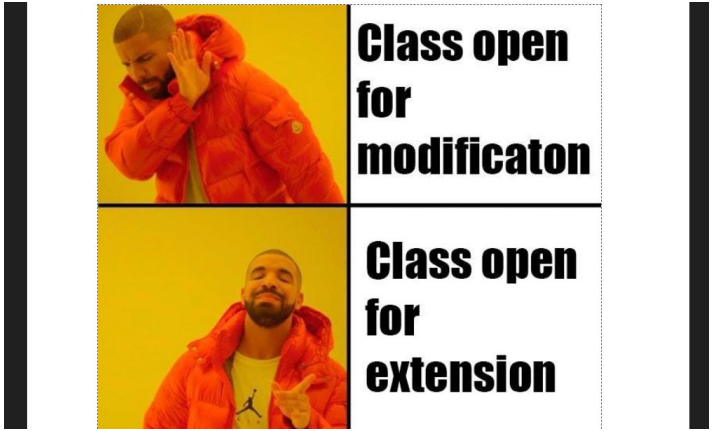
WHY?

- You focus on what is necessary:
- You can avoid bugs:
- Your code is more maintainable, testable, and flexible:


Rectangle
- width : int
- height : int
+ getWidth( ) : int
+ setWidth(int width) : void
+ getHeight( ) : int
+ setHeight(int height) : void
+ calculateArea( ) : int


<<Abstract>>
Shape
+ calculateArea( ) : abstract int
Extends
Rectangle
- width : int
- height : int
+ calculateArea( ) : int
Circle
- radius : int
+ calculateArea( ) : int

Identifying violations of OCP



1.Type Checking with instanceof or switch on Type

2. Long Chain of if-else Conditions

3. Frequent Modifications for New Cases

```java
// PROBLEM: This class must be modified for every new shape
public class AreaCalculator {
    public double calculateArea(Object shape) {
        if (shape instanceof Rectangle) {
            Rectangle rect = (Rectangle) shape;
            return rect.getWidth() * rect.getHeight();
        } else if (shape instanceof Circle) {
            Circle circle = (Circle) shape;
            return Math.PI * circle.getRadius() * circle.getRadius();
        }
        // What happens when we need to add Triangle?
        // We MUST modify this method!
        throw new IllegalArgumentException("Unknown shape type");
    }
}
```

SOLUTION?

Create Abstractions

```java
// Abstraction that defines the contract
public interface Shape {
    double calculateArea();
}
```

```java
public class Rectangle implements Shape {
    private double width;
    private double height;

    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    @Override
    public double calculateArea() {
        return width * height;
    }

    // getters and setters
}
```

```java
public class Circle implements Shape {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public double calculateArea() {
        return Math.PI * radius * radius;
    }

    // getters and setters
}
```

```java
// This class is NOW CLOSED for modification
public class AreaCalculator {
    public double calculateArea(Shape shape) {
        return shape.calculateArea(); // Delegates to the specific shape
    }
}
```

Common Misconceptions

Misconception 1: "Never Change Existing Code"

OCP doesn't mean you should never modify existing code. It means you shouldn't have to modify stable, working code to add new features.

When it's OK to modify:

- Bug fixes
- Performance improvements
- Refactoring for better design

Common Misconception

Misconception 2: "Apply OCP Everywhere From Day One"

This leads to over-engineering. Apply OCP when:

- You anticipate multiple variations
- The requirement is likely to change
- You're adding the second similar feature

L: Liskov Substitution Principle

"Objects of a superclass should be replaceable with objects of its subclasses without breaking the application."

Liskov Substitution Principle

Think of LSP as the "is-a" contract. When you say "Square is-a Rectangle", the Square must behave exactly like a Rectangle in every context where a Rectangle is expected. If substituting a Square for a Rectangle breaks your program, then the "is-a" relationship is invalid.

```java
// Base class
class Rectangle {
    protected int width;
    protected int height;

    public void setWidth(int width) {
        this.width = width;
    }

    public void setHeight(int height) {
        this.height = height;
    }

    public int getArea() {
        return width * height;
    }
}
```

```java
class Square extends Rectangle {

    @Override
    public void setWidth(int width) {
        this.width = width;
        this.height = width; // VIOLATION: Changes height too!
    }

    @Override
    public void setHeight(int height) {
        this.height = height;
        this.width = height; // VIOLATION: Changes width too!
    }
}
```

```java
public class LspTest {
    public static void main(String[] args) {
        // This function works with Rectangle and expects certain behavior
        void testRectangleArea(Rectangle rectangle) {
            rectangle.setWidth(5);
            rectangle.setHeight(10);

            // We expect area to be 5 * 10 = 50
            System.out.println("Expected area: 50, Actual area: " + rectangle.getArea());

            // This assertion should NEVER fail for any Rectangle
            assert rectangle.getArea() == 50 : "LSP Violation!";
        }
```

```
// Test with actual Rectangle - WORKS FINE
Rectangle rectangle = new Rectangle();
testRectangleArea(rectangle); // Output: Expected area: 50, Actual area: 50 ✅

// Test with Square - BREAKS!
Rectangle squareAsRectangle = new Square();
testRectangleArea(squareAsRectangle); // Output: Expected area: 50, Actual area: 100 ❌
// ASSERTION FAILS! LSP VIOLATION!
```

Identifying violations of LSP

1. Type Checking in Client Code
2. Overridden Methods That Throw Unexpected Exceptions
3. Overridden Methods with Completely Different Behavior
4. Empty Method Overrides
5. Changed Method Preconditions/Postconditions
7. Downcasting in Client Code

Common Misconception

What People Think Overriding Means:

"I can make the method do anything completely different"

What Overriding Actually Should Mean:

"I can provide a different implementation but the method must still fulfill the same contract"

I: Interface Segregation Principle

"Clients should not be forced to depend on interfaces they do not use."

Interface segregation

Think of ISP as the "specialized tools vs. Swiss Army knife" principle. Instead of one giant interface that does everything, create multiple focused interfaces.

Keep interfaces small and focused. Don't make clients implement methods they don't need

```java
// 🚨 VIOLATION: One interface for all media types
interface MediaPlayer {
    void playAudio();
    void playVideo();
    void displayImage();
    void readText();
}

class AudioPlayer implements MediaPlayer {
    public void playAudio() { /* works */ }
    public void playVideo() { throw new UnsupportedOperationException(); }
    public void displayImage() { throw new UnsupportedOperationException(); }
    public void readText() { throw new UnsupportedOperationException(); }
}
```

SOLUTION?

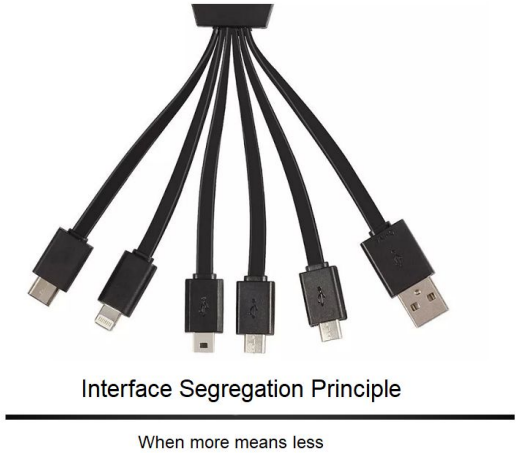
```java
// ✅ Segregated interfaces
interface AudioPlayer {
    void playAudio();
    void pauseAudio();
    void stopAudio();
}

interface VideoPlayer {
    void playVideo();
    void pauseVideo();
    void stopVideo();
}

interface ImageDisplay {
    void displayImage();
    void zoomImage();
}
```

```java
// Implement only what you need
class SimpleAudioPlayer implements AudioPlayer {
    public void playAudio() { /* implementation */ }
    public void pauseAudio() { /* implementation */ }
    public void stopAudio() { /* implementation */ }
}
```

Identifying Violations of ISP



1. Throwing UnsupportedOperationException
2. Empty Method Implementations
3. Interface with Too Many Methods
4. Clients Using Only Subsets of Interface

Common Misconception

Misconception 1: "ISP Only Applies to Interfaces"

Wrong Belief: "ISP is only about Java interfaces or C# interfaces."

Reality: ISP applies to any API contract - including abstract classes, service contracts, REST APIs, etc.

Misconception 2: "ISP is Just SRP for Interfaces"

Wrong Belief: "ISP is the same as Single Responsibility Principle but for interfaces."

Reality: They're related but distinct.

Common Misconception

Misconception 3: "ISP Means Never Having Large Interfaces"

Wrong Belief: "Any interface with more than X methods violates ISP."

Reality: It's about client needs, not method count.

Misconception 4: "ISP Makes Code More Complex"

Wrong Belief: "More interfaces = more complexity."

Reality: Proper ISP reduces complexity in the long run.

Common Misconception

Misconception 5: "ISP Means No Method Can Be Used by Multiple Clients"

Wrong Belief: "If two different clients use the same method, I must split the interface."

Reality: ISP is about unused methods, not shared ones.

Misconception 6: "ISP Requires Creating Interfaces for Every Possible Client"

Wrong Belief: "I need to anticipate all client needs and create specialized interfaces upfront."

Reality: Apply ISP when you need it, not preemptively.

D: Dependency Inversion Principle

"High-level modules should not depend on low-level modules. Both should depend on abstractions."

Dependency Inversion Principle

This aims to reduce the coupling between the classes is achieved by introducing abstraction between the layer, thus doesn't care about the real implementation.

```java
public class OrderService {
    // Direct dependency on concrete implementations
    private MySQLDatabase database;
    private SMTPEmailService emailService;
    private StripePaymentGateway paymentGateway;

    public OrderService() {
        this.database = new MySQLDatabase();          // ❌ Tight coupling
        this.emailService = new SMTPEmailService();   // ❌ Tight coupling
        this.paymentGateway = new StripePaymentGateway(); // ❌ Tight coupling
    }

    public void processOrder(Order order) {
        database.save(order);                     // ❌ Direct dependency
        paymentGateway.chargeCreditCard(order);   // ❌ Direct dependency
        emailService.sendConfirmation(order);     // ❌ Direct dependency
    }
}
```

SOLUTION?

```java
public class OrderService {
    private OrderRepository orderRepository;
    private PaymentGateway paymentGateway;
    private NotificationService notificationService;

    // ✅ DEPENDENCY INJECTION: Dependencies provided from outside
    public OrderService(OrderRepository repository,
                        PaymentGateway gateway,
                        NotificationService notifier) {
        this.orderRepository = repository;
        this.paymentGateway = gateway;
        this.notificationService = notifier;
    }

    public void processOrder(Order order) {
        // Business logic remains unchanged
        orderRepository.save(order);
        paymentGateway.processPayment(order);
        notificationService.sendOrderConfirmation(order);
    }
```

```java
// ✅ COMPOSITION ROOT: Wire everything together at application startup
public class Application {
    public static void main(String[] args) {
        // Choose implementations (could be based on configuration)
        OrderRepository repository = new MySQLOrderRepository();
        PaymentGateway gateway = new StripePaymentGateway();
        NotificationService notifier = new SMTPNotificationService();

        // Inject dependencies
        OrderService orderService = new OrderService(repository, gateway, notifier);

        // Use the service
        Order order = new Order();
        orderService.processOrder(order);
    }
}
```

Common Misconception

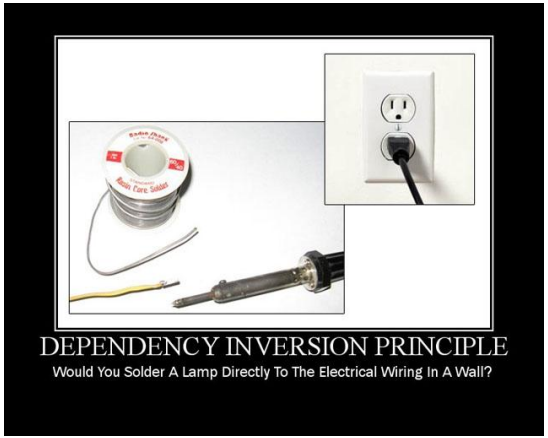
Misconception 1: "Every class needs an interface"

Reality: Only create interfaces for dependencies that might change or have multiple implementations.

Misconception 2: "DIP makes simple code complex"

Reality: DIP pays off in medium to large applications where change is inevitable.

Identifying DIP violations



1. Direct dependency on concrete implementations through static methods.
2. "new" Keyword for Dependencies

Let's Look at some real-life Code